PRAKTIKUM
SISTEM BASIS DATA

Nama	**Jonathan Frederick Kosasih**	**No. Modul**	**6**
NPM	**2306225981**	**Tipe**	**CS**

1. Regex yang digunakan untuk memeriksa validitas email dan password:

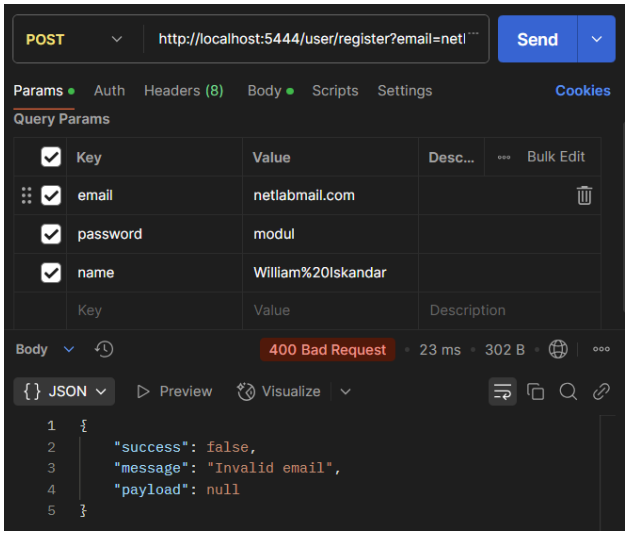
```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const passRegex = /^(?=.*[^\s]{8,})(?=.*\d)(?=.*[^\w\d]).+$/;
```

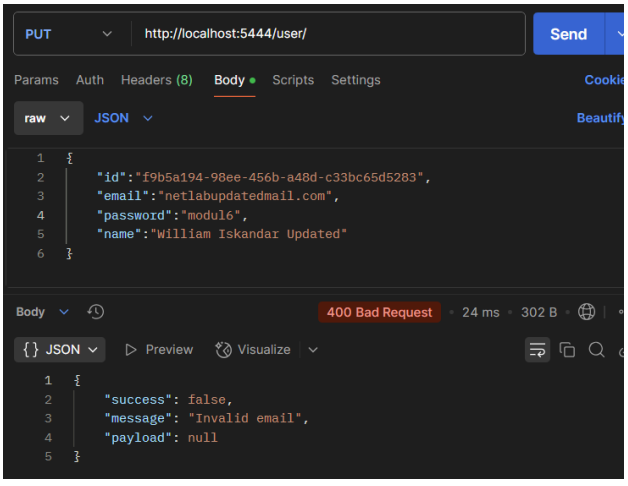
Perubahan kode untuk mengimplementasi regex:

```
exports.registerUser = async (req, res) => {
    if (!req.query.name || !req.query.email || !req.query.password) {
        return baseResponse(res, false, 400, "Missing user name, email, or password",
        null);
    }
    try {
        if (!emailRegex.test(req.query.email)) {
            return baseResponse(res, false, 400, "Invalid email", null);
        }
        if (await userRepository.getUserByEmail(req.query.email)) {
            return baseResponse(res, false, 400, "Email already registered", null);
        }
        if (!passRegex.test(req.query.password)) {
            return baseResponse(res, false, 400, "Password must be at least 8 characters
            long, contain at least one number, and one special character", null);
        }
        const hashedPassword = await bcrypt.hash(req.query.password, saltRounds);
        const user = await userRepository.registerUser(req.query.name, req.query.email,
        hashedPassword);
        baseResponse(res, true, 201, "User created", user);
    } catch (error) {
        baseResponse(res, false, 500, error.message || "Server error", error);
    }
}
```

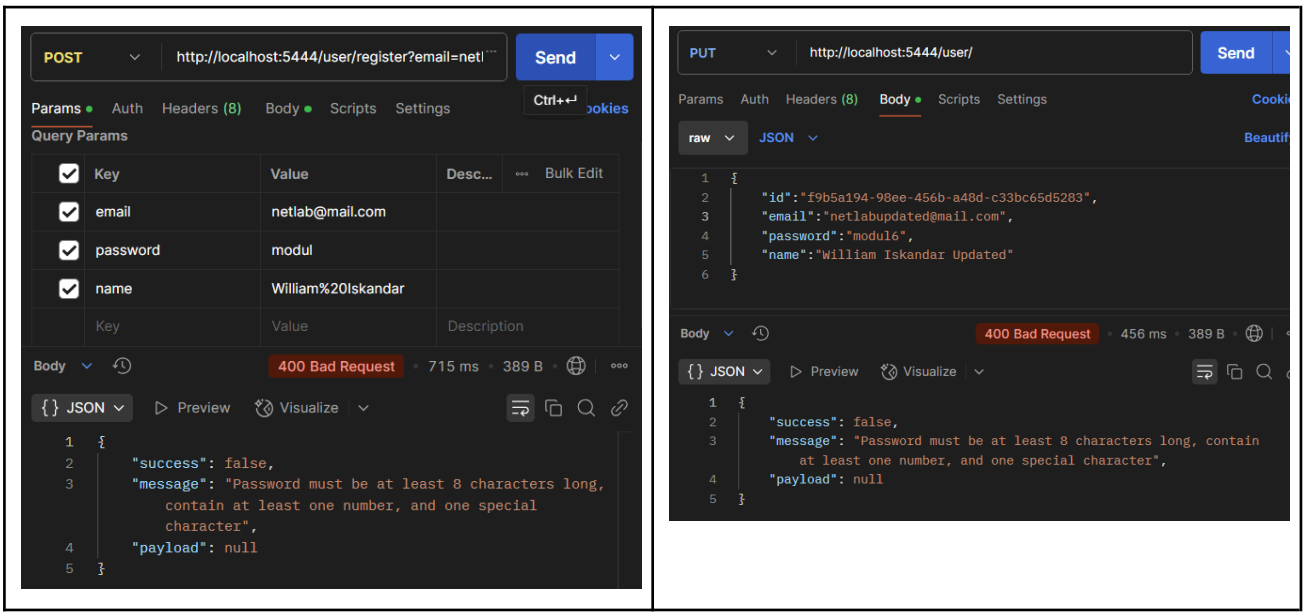
```
exports.updateUser = async (req, res) => {
    if (!req.body.name || !req.body.email || !req.body.password) {
        return baseResponse(res, false, 400, "Missing user name, email, or password",
        null);
    }
    try {
        if (!emailRegex.test(req.body.email)) {
            return baseResponse(res, false, 400, "Invalid email", null);
        }
        if (await userRepository.getUserByEmail(req.body.email)) {
            return baseResponse(res, false, 400, "Email already registered", null);
        }
        if (!passRegex.test(req.body.password)) {
            return baseResponse(res, false, 400, "Password must be at least 8 characters
            long, contain at least one number, and one special character", null);
        }
        const hashedPassword = await bcrypt.hash(req.body.password, saltRounds);
        const user = await userRepository.updateUser(req.body.name, req.body.email,
        hashedPassword, req.body.id);
        if (!user) {
            return baseResponse(res, false, 404, "User not found", null);
        }
        baseResponse(res, true, 200, "User updated", user);
    } catch (error) {
        baseResponse(res, false, 500, "Error updating user", error);
    }
}
```

Screenshot ketika email tidak sesuai dengan regex:

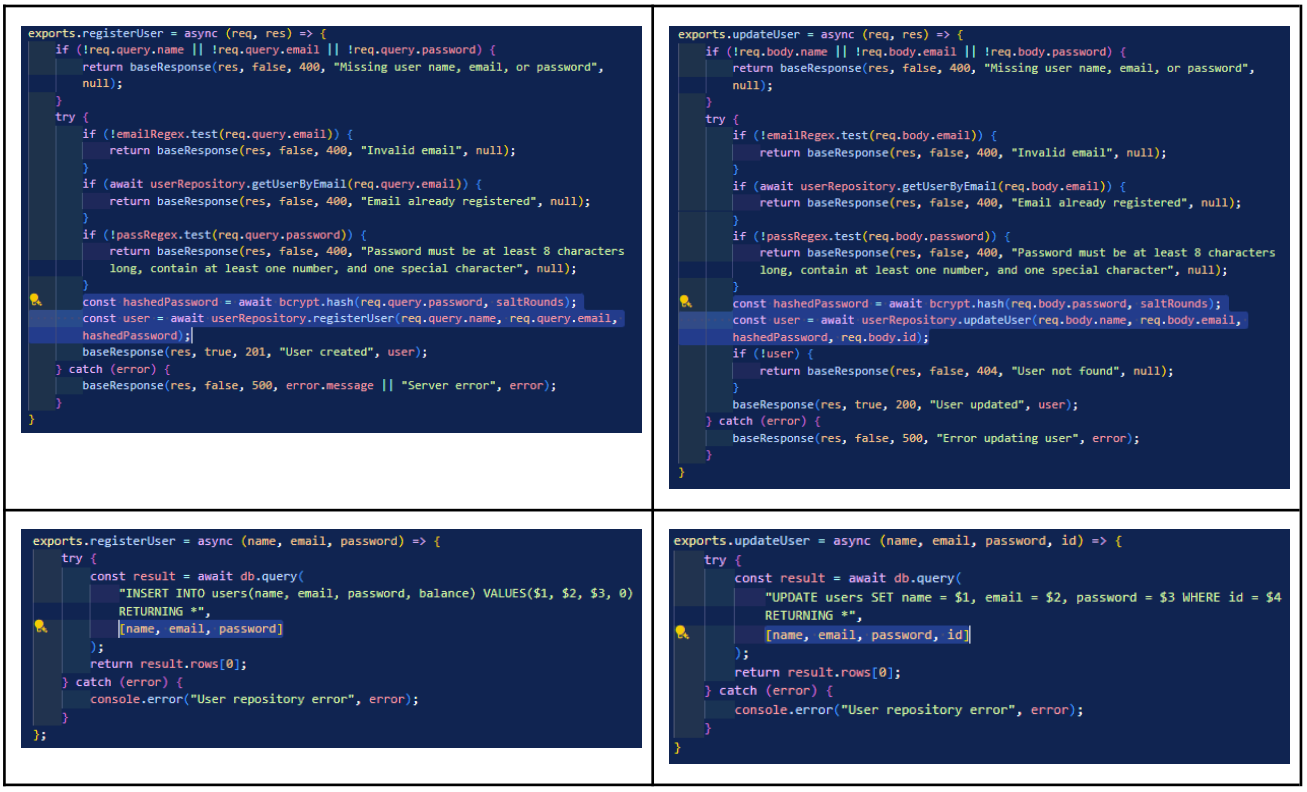




Screenshot ketika password tidak sesuai dengan regex:



2. Perubahan kode untuk mengimplementasikan hash password:



3. Perubahan kode untuk mengimplementasi compare hash password:

```
exports.loginUser = async (email, password) => {
    try {
        const result = await db.query("SELECT * FROM users WHERE email = $1", [email]);
        const compare = await bcrypt.compare(password, result.rows[0].password);
        if (!compare) {
            return null;
        }
        else {
            return result.rows[0];
        }
    } catch (error) {
        console.error("User repository error", error);
    }
}
```

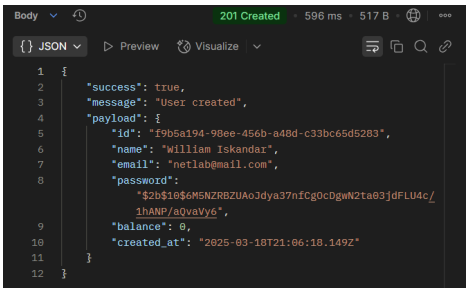
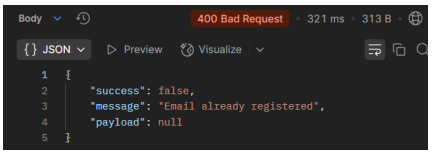
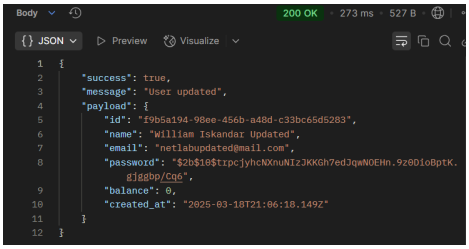
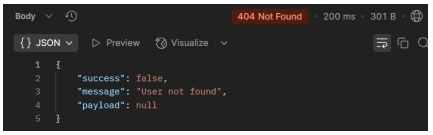
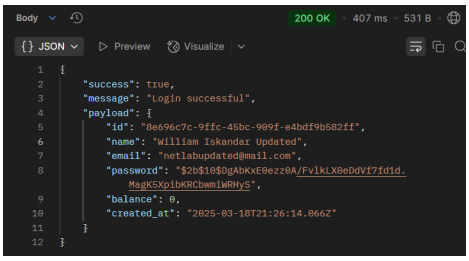
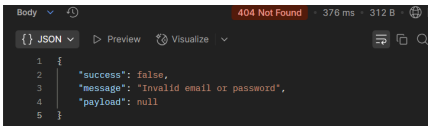
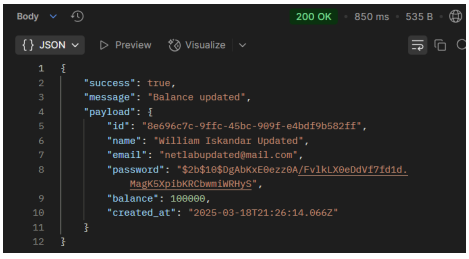
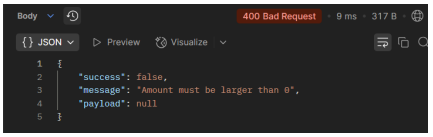
4. Perubahan kode untuk mengimplementasi CORS:

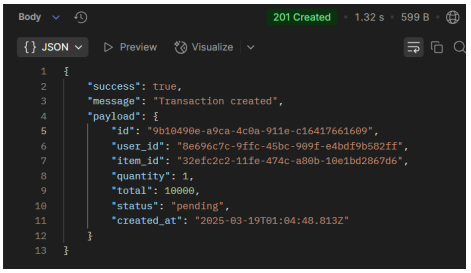
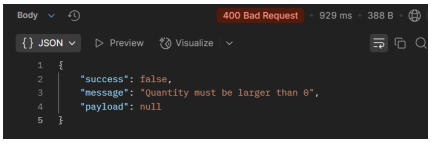
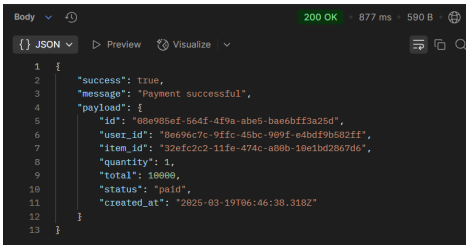
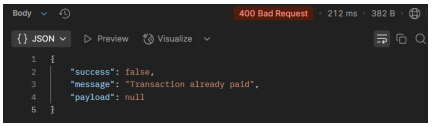
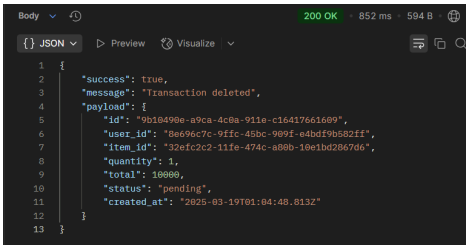
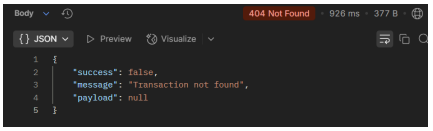
```
JS server.js > ...
1   const express = require('express');
2   const cors = require('cors');
3   require('dotenv').config();
4
5   const app = express();
6   const port = process.env.PORT || 3000;
7   const corsOptions = {
8       origin: "https://os.netlabdte.com",
9       methods: ["GET", "POST", "PUT", "DELETE"],
10  };
11
12  app.use(cors(corsOptions));
13  app.use(express.json());
```

5. Screenshot menjalankan query:

```
prak-sbd=> CREATE type transaction_status AS ENUM ('pending', 'paid');
CREATE TYPE
prak-sbd=> CREATE TABLE IF NOT EXISTS transactions(
prak-sbd(>  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
prak-sbd(>  user_id UUID NOT NULL,
prak-sbd(>  item_id UUID NOT NULL,
prak-sbd(>  quantity INT NOT NULL,
prak-sbd(>  total INT NOT NULL,
prak-sbd(>  status transaction_status DEFAULT 'pending',
prak-sbd(>  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
prak-sbd(>  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
prak-sbd(>  FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
prak-sbd(> );
CREATE TABLE
prak-sbd=>
```

6. Pada CS kali ini, saya merevisi beberapa file dan menambahkan beberapa file, di antaranya transaction.repository.js, transaction.controller.js, transaction.route.js. Source code untuk ketiga file baru dan perubahan file lainnya terlampir pad a.zip file yang dikumpulkan.
7. Screenshot:

Method	Endpoint	Success	Failed
POST	/user/register		
PUT	/user		
POST	/user/login		
POST	/user/topUp		

POST	/transaction /create		
POST	/transaction /pay		
DELETE	/transaction /:id		

8. Screenshot directory (sebelum memasukkan PDF CS)